Workflow Diagrams

Wladimir E. Banda Barragán

wbanda@yachaytech.edu.ec

Yachay Tech University - 2025-2026

Crash course on Python

Python is a high-level, interpreted programming language.

It uses indentation to define code blocks.

In Python, **everything is an object.** Objects are at the code of the language and are self-contained containers.

Object-Oriented Programming (OOP) is a method for designing software that organises the code into containers that group both data (attributes) and functions (methods) that operate on that data into a single unit.

The data and the functions that manipulate that data are tightly bundled together within an object.

Objects in Python

Mutable Objects: objects whose state can be changed after they are created. We can modify, add, or remove elements from these objects without creating a new object in memory).

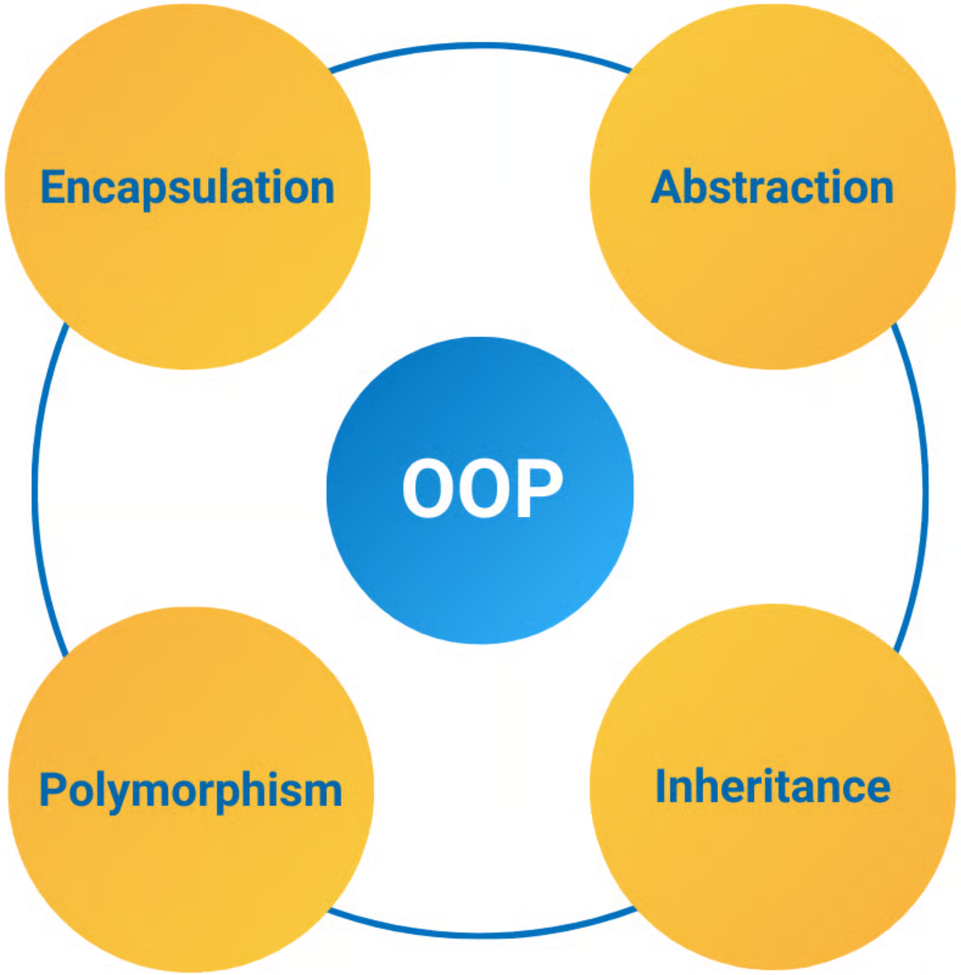
Examples: list, dict, set

Immutable Objects: objects whose state cannot be changed after they are created. Any operation that appears to modify an immutable object actually creates a new object in memory.

Examples: int, float, str, tuple, function, class

OOP helps to create modular, reusable, and more maintainable code. It makes it easier to manage large, complex programs by breaking them down into smaller, self-contained components.

Object-oriented Programming (OOP)



Encapsulation: Merging data **(attributes)** and the functions **(methods)** that operate on that data into a single unit (an object). This hides the internal state of an object from the outside world.

Abstraction: Hiding complex implementation details and showing only the essential features of an object. This simplifies the user's interaction with the object.

Polymorphism: The ability of objects of different classes to respond to the same method call in their own unique way. The word "polymorphism" means "many forms."

Inheritance: A mechanism where a new class (**derived class**) can inherit properties and behaviours from an existing class (**main class**). This promotes code reuse.

Functions in Python

def/return functions: Customised functions.

```
# Header
def thermal_pressure(nden, temp):
  # Body
  # Docstring:
  """
  Function computes the thermal pressure of ideal gases.
  Inputs: nden (number density), temp (temperature)
  Output: prs (pressure)
  Author: W.E.B.B.
  Date created: 28/04/23
  Date modified: 26/02/2024
  """

  # What you compute
  prs = nden*k_b*temp

  # What you return
  return prs
```

Header & Arguments

Docstring

Accesed via help()

Return Statement

Functions in Python

Lambda Functions: These are used to quickly define and use functions.

```
# Here we can use a lambda function
z_1D = lambda x: x**3
```

1-line Statement

Nested & Iterative Functions

```
def func1():
  “”” Local Variable Access “””
  msg = “Local Variable”
  def func2():
    print(msg)

  func2()

func1()
```

```
def factorial(n):
  """
  Calculate and return the factorial of n.
  """
  if n == 1 or n==0:
    # First case
    f_1 = 1
    return f_1
  else:
    # Other cases, we do a recursive call
    f_2 = n*factorial(n-1)
    return f_2
```

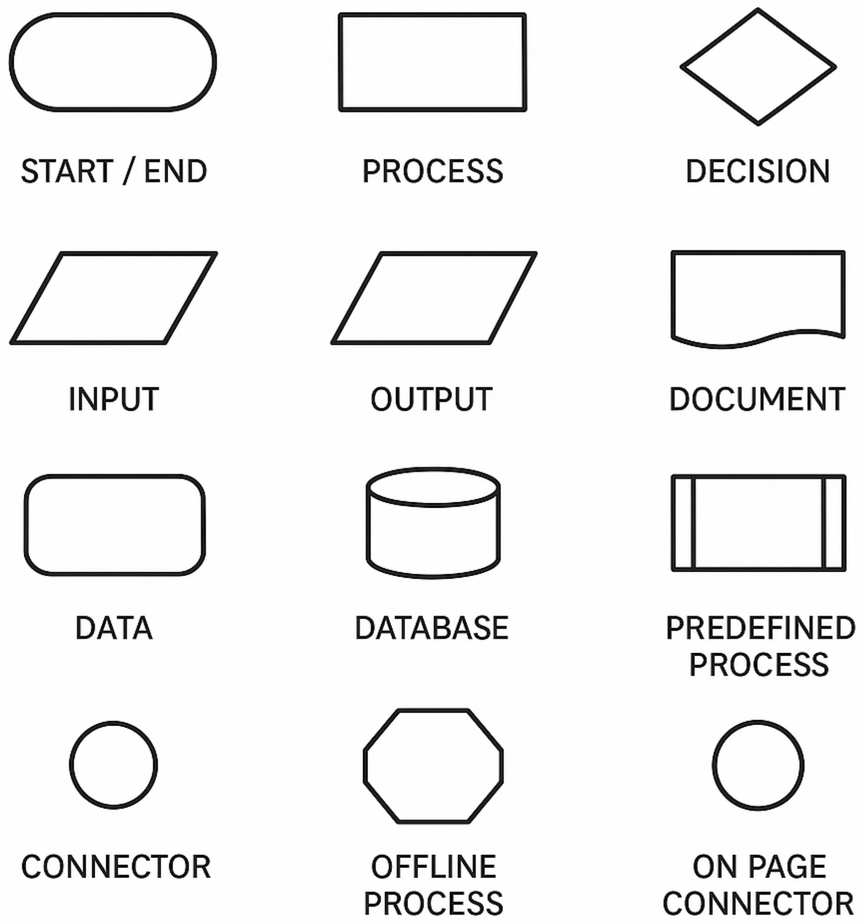
OOP Algorithms and Code WorkFlows

Code workflows are essential for software design as they:

- Separate logical planning from complex programming syntax.
- Create a universal visual language for code developing teams.
- Make complex processes and dependencies easier to understand.
- Help identify logical errors and optimise flow before coding.
- Provide intuitive, lasting documentation for any system.
- Demonstrate a true understanding of a problem's structure and solution.

OOP Algorithms and Code WorkFlows

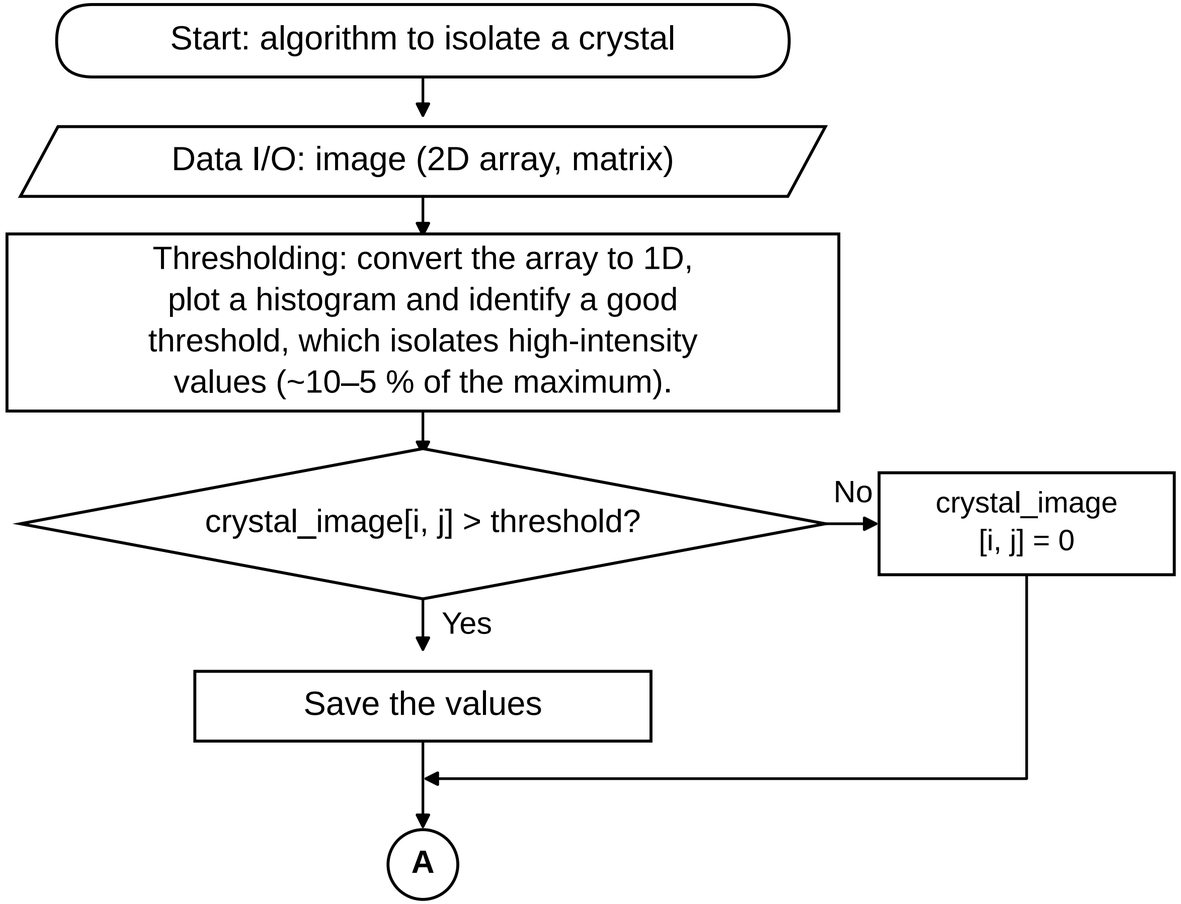
Usual symbols include:

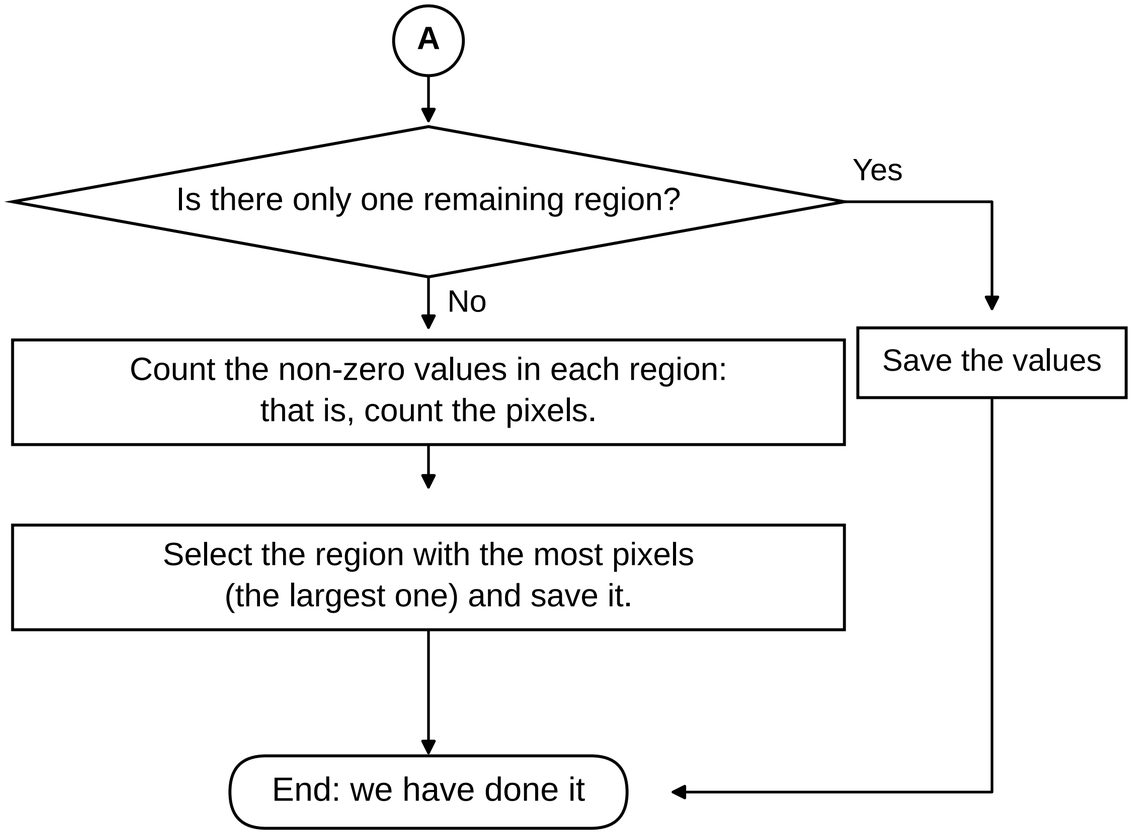


ANSI Standard:
https://nvlpubs.nist.gov/nistpubs/Legacy/FIPS/fipspub24.pdf

Example: isolating a crystal

You obtain a photograph of iron crystals and are asked to isolate the most prominent one from the background and from the rest of the image.





Solution developed with Britney Robalino Ramírez.

Isolating a crystal: tools and methods

Thresholding

```
# 1D: .reshape() or .flatten()
# test several thresholds; plt.imshow() with a colorbar is handy
```

The comparison

```
# [i, j] run over every pixel in the image
# np.where() works here, since it accepts conditions
```

Counting the pixels

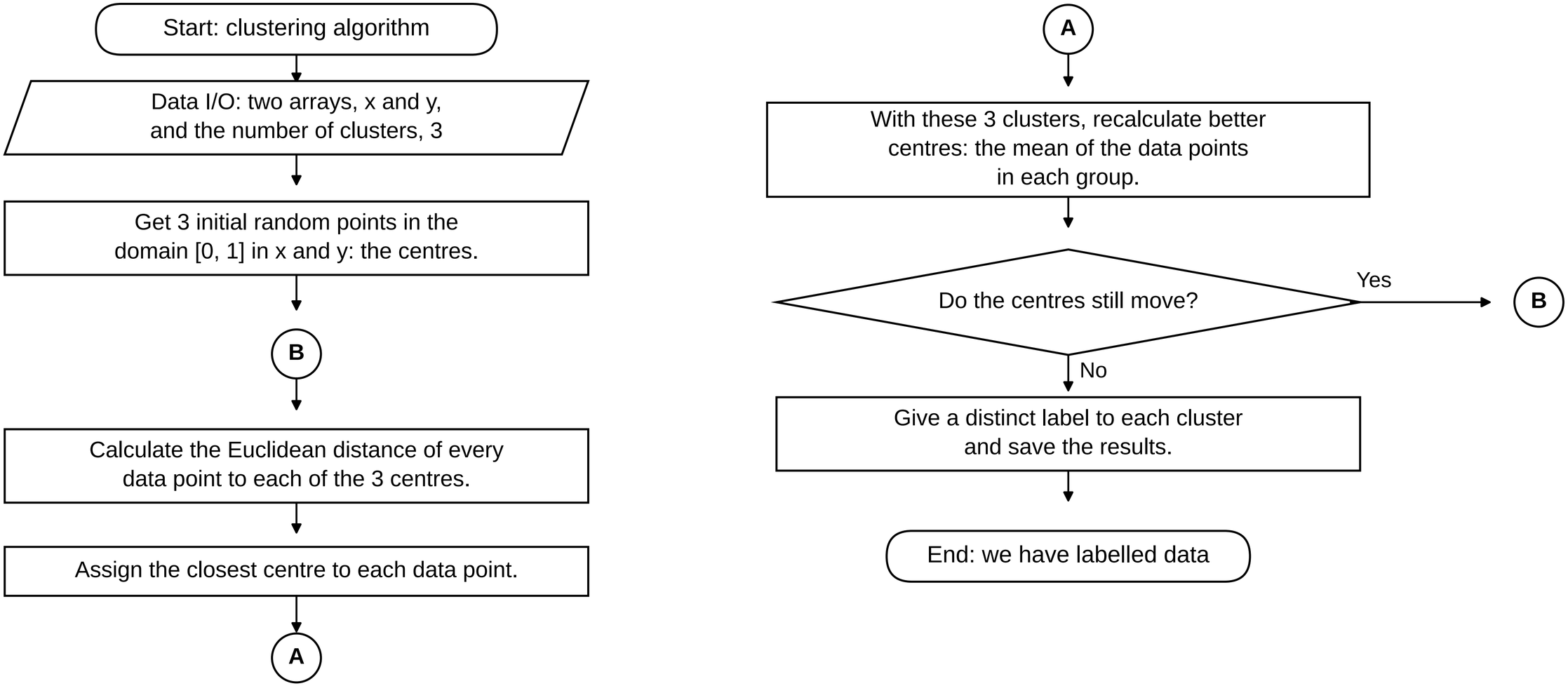
```
# convert the image to binary [0 1], then np.sum() gives
#   the number of pixels in a region
```

Selecting the region

```
# with many regions, np.max() picks the most prominent
```

Example: a clustering algorithm

Given a distribution of points, how would you give a distinct label to each of the three apparent groups, so that points with the same label belong to the same group?



Clustering: tools and methods

The initial centres

```
# np.random.rand() returns values in [0, 1]
# two draws per centre, one for x and one for y
```

The distances

```
# np.linalg.norm() does this, and is remarkably fast
# the norm has to be specified
```

The assignment

```
# np.argmin() gives the position of the smallest distance
```

The new centres

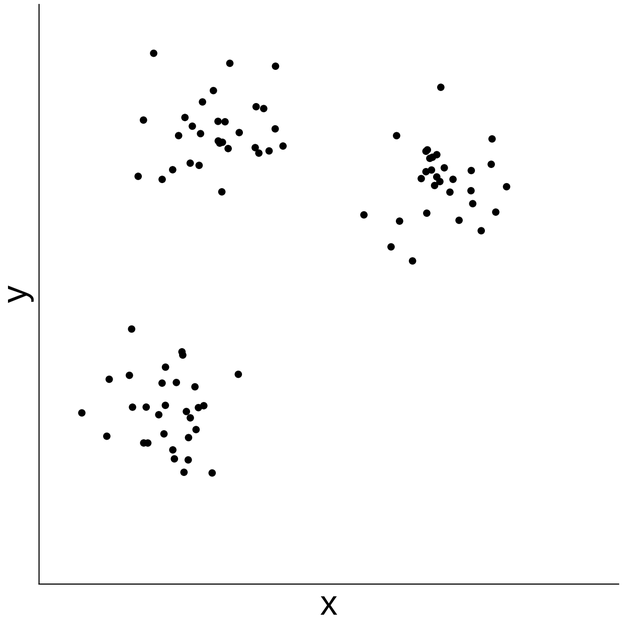
```
# np.mean() over the points of each group
```

Convergence

```
# define an epsilon (eps = 1e-6) to decide whether they still move
# a while loop on the difference between centres can be used
```

The labels

```
# the labels can simply be the numbers [0 1 2]
```



The three apparent groups

WorkFlow Editors

There are a few (free) workflow editors:

1. Raptor

RAPTOR is a flowchart-based programming environment, designed specifically to help students visualise their algorithms and avoid syntactic baggage.

It is free and allows to visualise logic and create executable flowcharts.

Link: https://raptor.martincarlisle.com

2. yEd Graph Editor

yED is used for creating large, complex, and professional-looking diagrams.

yED works offline.

Link: https://www.yworks.com/products/yed

WorkFlow Editors

There are a few (free) workflow editors:

3. Diagrams.net (draw.io)

Diagrams.net is a web-based tool to create workflow diagrams with flowchart shapes.

It is free, open-source and integrates with Google Drive, OneDrive, and GitHub.

Link: https://app.diagrams.net/

4. Code2Flow

Also web-based with a free tier (limited number of charts per day.)

You can generate flowcharts from existing code.

Link: https://code2flow.com/